0746. Min Cost Climbing Stairs

Easy · 1-D Dynamic Programming

leetcode.com/problems/min-cost-climbing-stairs

Problem

- Cost[i] is paid when you step on stair i; you may then climb 1 or 2. Start at index 0 or 1. Return the minimum cost to reach the top (index n).

Intuition

- Dp[i] = minimum cost to stand on i (before paying for it) = min(dp[i-1] + cost[i-1], dp[i-2] + cost[i-2]).

Brute Force

- Recursion over both choices: O(2^n).

Optimal Approach

- A = dp[0] = 0, b = dp[1] = 0.
- For i = 2..n: c = min(b + cost[i-1], a + cost[i-2]); shift.
- Return b.

Worked Example

```
cost = [10,15,20].

dp[0]=0 dp[1]=0
dp[2]=min(0+15, 0+10)=10
dp[3]=min(10+20, 0+15)=15   -> answer 15
```

Java Solution

```java
public class MinCostClimbingStairs {
    public int minCostClimbingStairs(int[] cost) {
        int a = 0, b = 0;
        for (int i = 2; i <= cost.length; i++) {
            int c = Math.min(b + cost[i - 1], a + cost[i - 2]);
            a = b;
            b = c;
        }
        return b;
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- The top is index n (past the last stair).